Computer Vision (Modules 4-6)
Complete Study Material

Audit-Ready Study Guide

Contents

1 Module 4: Generative Adversarial Networks (GANs)

1.1 Introduction to GANs

- **Definition**: A Generative Adversarial Network (GAN) is a deep learning architecture consisting of two competing neural networks: a **Generator** and a **Discriminator**.
- **Inventor**: Ian Goodfellow (2014).
- **Analogy**: Like a forger (Generator) creating fake currency and a detective (Discriminator) trying to catch the fakes. Through competition, both improve until the fakes are indistinguishable from reality.

1.2 Core Architecture and Process

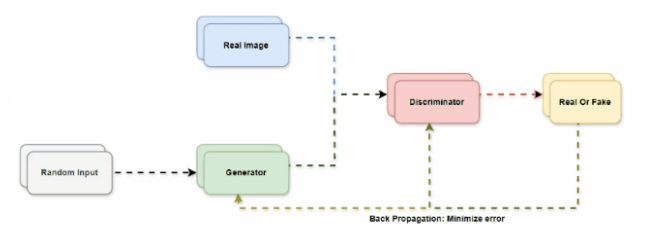

Figure 1: General GAN Architecture

- **Generator (G)**:
  - **Input**: Random noise (latent vector).
  - **Output**: Fake data (e.g., generated images).
  - **Goal**: Produce data so realistic that it fools the Discriminator.
- **Discriminator (D)**:
  - **Input**: Real data (from the training set) and Fake data (from the Generator).
  - **Output**: Probability score (0 = Fake, 1 = Real).
  - **Goal**: Accurately distinguish between real and fake data.

Training Loop (Min-Max Loss)

1. **Noise Sampling**: Generator takes a random noise vector.
2. **Data Generation**: Generator creates a fake image.
3. **Discrimination**: Discriminator evaluates the fake image against a real image.
4. **Adversarial Training**: Feedback (loss) is sent via backpropagation. The Generator updates weights to minimize the discriminator's accuracy, while the Discriminator updates weights to maximize its accuracy.

1.3 The Role of the Noise Vector

Why use random noise as input to the Generator?

- **Generate Variety (Diversity)**: Without noise, the generator would produce the exact same output every time. Noise acts as a seed for variation.
- **Learn Data Distribution**: The noise maps a simple distribution (like Gaussian) into a complex, high-dimensional real-data distribution.
- **No Bias**: Noise lacks predefined structures, forcing the model to learn pure data patterns.

1.4 Implicit vs. Explicit Density Estimation

- **Implicit Density (GANs)**: Learns to generate data that *resembles* the real data distribution without ever explicitly calculating the exact probability distribution.
- **Explicit Density (VAEs - Variational Autoencoders)**: Explicitly learns and computes the probability distribution of data using likelihood (like following an exact recipe).

1.5 Limitations of Vanilla GANs

1. **Mode Collapse**: The Generator finds a single output (or a small set of outputs) that easily fools the Discriminator and stops producing diverse samples. (e.g., generating the exact same face every time).
2. **Training Instability**: If the Discriminator gets too strong too quickly, the Generator receives weak gradients and stops learning. If the Generator is too strong, the Discriminator fails.
3. **Vanishing Gradients**: Saturation in the Discriminator's sigmoid activation function can halt the Generator's learning. (Solved partially using Leaky ReLU or different loss functions).
4. **Difficult Hyperparameter Tuning**: Highly sensitive to learning rates, batch sizes, and architecture.

1.6 Advanced GAN Variants

- **CGAN (Conditional GAN)**: Takes an additional *label* as input. The label dictates the *category* (e.g., "rose"), while the noise vector dictates the *variation* (shape, color).
- **DCGAN (Deep Convolutional GAN)**: Replaces fully connected layers with Convolutional Neural Networks (CNNs). Uses Strided Convolutions (instead of pooling), Batch Normalization, and Leaky ReLU.

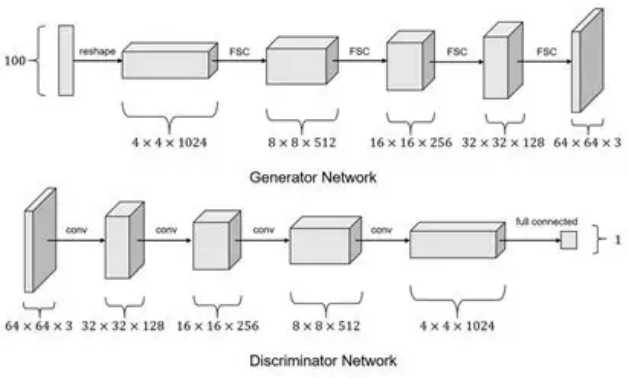

Figure 2: DCGAN Generator Architecture

- **WGAN (Wasserstein GAN)**: Replaces standard divergence loss with the Wasserstein distance (Earth Mover's Distance) to measure differences between real/fake, drastically improving training stability.
- **ProGAN (Progressive GAN)**: Starts generating low-resolution images and progressively adds layers to increase resolution and detail.
- **StyleGAN**: Allows fine-grained control over the generated image's style (e.g., pose, lighting, texture).

1.7 Solved Numericals: Module 4

Numerical 1: Discriminator Loss Calculation

Given:

- $D(x) = 0.9$ (Discriminator's probability output for a real image)
- $D(G(z)) = 0.3$ (Discriminator's probability output for a fake image)

Objective: Calculate the Discriminator's Loss.
Steps:

1. **Formula:** The Discriminator Loss (using natural logarithm) is defined as:

$$\text{Loss}_D = -[\ln(D(x)) + \ln(1 - D(G(z)))]$$

2. **Substitute values:**

$$\text{Loss}_D = -[\ln(0.9) + \ln(1 - 0.3)]$$
$$\text{Loss}_D = -[\ln(0.9) + \ln(0.7)]$$

3. **Calculate logarithms:**

$$\ln(0.9) \approx -0.105, \quad \ln(0.7) \approx -0.357$$

4. **Final calculation:**

$$\text{Loss}_D = -(-0.105 - 0.357) = -(-0.462) = 0.462$$

Interpretation: A lower loss indicates the Discriminator is doing well. 0.462 is relatively low, meaning it correctly identified the real image (0.9 is close to 1) and the fake image (0.3 is close to 0).

Numerical 2: KL and JS Divergence

Given:

- Real Distribution $P = [0.5, 0.3, 0.2]$
- Predicted Distribution $Q = [0.2, 0.2, 0.6]$

Objective: Compute Kullback-Leibler (KL) Divergence and Jensen-Shannon Divergence (JSD).

Part 1: KL Divergence ($D_{KL}(P||M)$ and $D_{KL}(Q||M)$)

1. **Find average distribution M:**

$$M = \frac{1}{2}(P + Q) = \left[\frac{0.5 + 0.2}{2}, \frac{0.3 + 0.2}{2}, \frac{0.2 + 0.6}{2}\right] = [0.35, 0.25, 0.4]$$

2. **Compute $D_{KL}(P||M)$:**

$$D_{KL}(P||M) = \sum P(x) \ln\left(\frac{P(x)}{M(x)}\right)$$

- $0.5 \ln(0.5/0.35) = 0.5 \ln(1.428) = 0.178$
- $0.3 \ln(0.3/0.25) = 0.3 \ln(1.2) = 0.0546$
- $0.2 \ln(0.2/0.4) = 0.2 \ln(0.5) = -0.1386$

$$D_{KL}(P||M) = 0.178 + 0.0546 - 0.1386 = 0.094$$

3. **Compute $D_{KL}(Q||M)$:**

$$D_{KL}(Q||M) = \sum Q(x) \ln\left(\frac{Q(x)}{M(x)}\right)$$

- $0.2 \ln(0.2/0.35) = -0.112$
- $0.2 \ln(0.2/0.25) = -0.0446$
- $0.6 \ln(0.6/0.4) = 0.243$

$$D_{KL}(Q||M) = -0.112 - 0.0446 + 0.243 = 0.0864$$

Part 2: Jensen-Shannon Divergence (JSD)

1. **Formula:**

$$\text{JSD}(P||Q) = \frac{1}{2}D_{KL}(P||M) + \frac{1}{2}D_{KL}(Q||M)$$

2. **Calculate:**

$$\text{JSD}(P||Q) = \frac{1}{2}(0.094 + 0.0864) = \frac{0.1804}{2} = 0.0902$$

Interpretation: JSD is symmetric and always finite ($0 \leq JSD \leq 1$ with base 2). Here, JSD ≈ 0.0902, indicating a low-to-moderate difference between the distributions.

2 Module 5: Object Segmentation

2.1 Introduction to Segmentation

- **Definition**: Grouping pixels with similar visual characteristics into meaningful segments.
- **Resolution & DPI**: 1024×768 resolution = 786,432 pixels. DPI (Dots Per Inch) determines physical print size/quality, while PPI (Pixels Per Inch) is for screens.

2.2 Types of Segmentation

- **Semantic Segmentation**: Classifies every pixel into a category (e.g., all cars get the same "car" label). Cannot differentiate between two distinct cars next to each other.
- **Instance Segmentation**: Detects individual object instances. If there are two cars, they are labeled as "Car 1" and "Car 2".
- **Panoptic Segmentation**: Combines both. Uses the concepts of **Things** (countable objects with boundaries, like cars and people) and **Stuff** (uncountable regions without strict boundaries, like sky, grass, road).

2.3 Convolutional Receptive Fields & Pyramids

- **Receptive Field**: The area of the input image that a neuron "sees". Shallow layers see small patches (high resolution, edge details). Deep layers see large regions (low resolution, strong semantic meaning).
- **Image Pyramid**: Manually resizing the image multiple times to detect objects at different scales (computationally expensive).

2.4 Key Segmentation Architectures

- **FPN (Feature Pyramid Network)**: Solves multi-scale detection without multiple image passes.
  - **Bottom-Up Pathway**: Normal CNN extracting features (spatial resolution drops, semantic meaning increases).
  - **Top-Down Pathway**: Upsamples deep, low-resolution layers.
  - **Lateral Connections**: Combines the upsampled deep layers with corresponding shallow layers using 1x1 convolutions. Merges semantic strength with spatial detail.

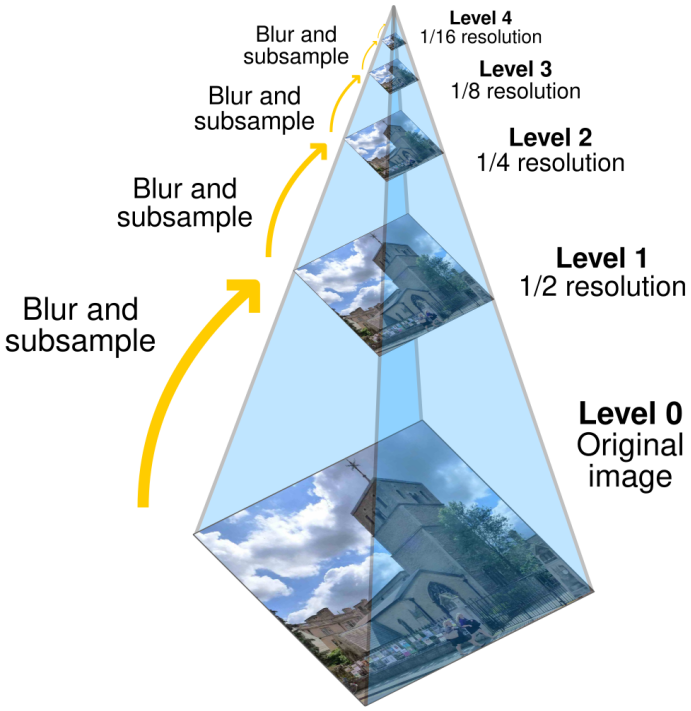

Figure 3: FPN Architecture

- **PSPNet (Pyramid Scene Parsing Network)**:
  - Uses a **Pyramid Pooling Module** that divides the feature map into different grid sizes (1×1, 2×2, 3×3, 6×6) to capture local and global contexts simultaneously. These are upsampled and concatenated before final pixel-wise softmax prediction.

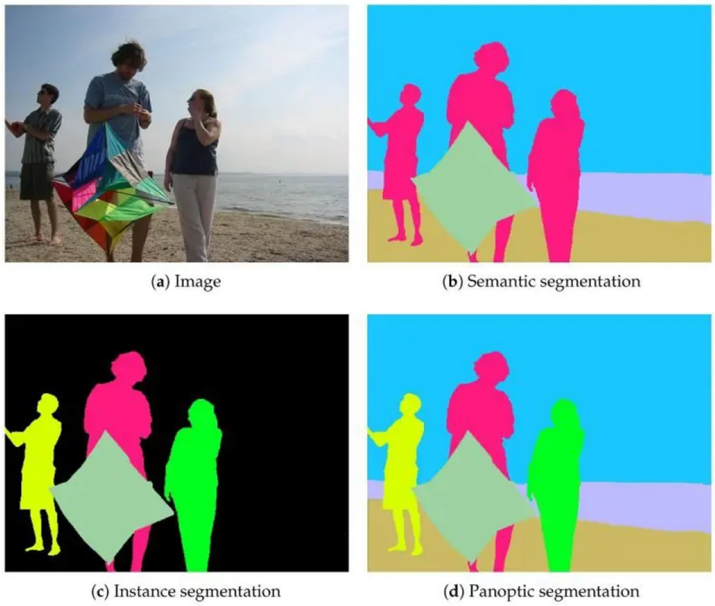

Figure 4: PSPNet Architecture

- **U-Net** (Widely used in Medical Imaging like Tumor Detection):
  - **Contracting Path (Encoder)**: Uses convolutions and pooling to capture context

(reduces spatial size).

- **Expansive Path (Decoder)**: Uses transposed convolutions to upsample and restore spatial resolution.
- **Skip Connections**: Crucial feature. Bypasses the deeper layers to directly pass high-resolution spatial details (like edges/boundaries) from the encoder to the decoder, preventing the loss of localization accuracy.

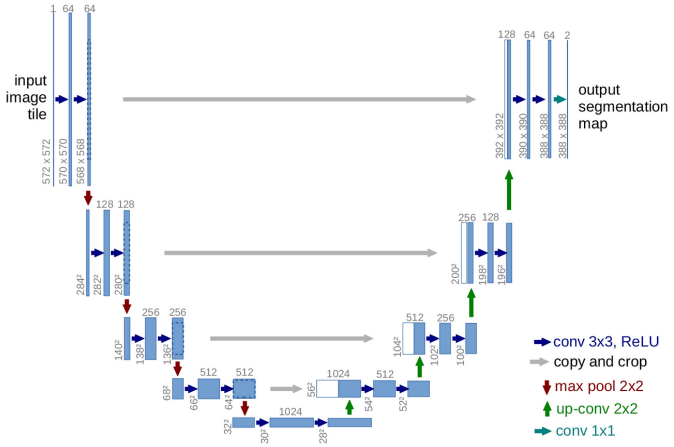

Figure 5: U-Net Architecture

2.5 Clustering Methods for Segmentation

- **K-Means**: Groups pixels into K predefined clusters based on similarity (minimizing variance).
- **Mean Shift**: Does not require predefined K. Finds clusters by moving data points towards areas of highest density.
- **DBSCAN**: Density-based. Groups closely packed pixels and isolates outliers/noise. Good for irregular shapes.
- **Agglomerative Hierarchical**: Builds a hierarchy by merging smaller clusters into larger ones. Relies on **Linkage Criteria**: Single (shortest distance), Complete (longest distance), Average, or Centroid.

2.6 Distance Metrics

- **Euclidean**: Straight-line distance (Pythagorean).
- **Manhattan**: Grid-like distance (sum of absolute differences).
- **Cosine Similarity**: Measures the angle between vectors (ignores magnitude).
- **Minkowski**: Generalized distance formula containing Euclidean and Manhattan.
- **Jaccard Index / Sorensen-Dice**: Measures overlap/similarity between two sets.

2.7 Solved Numericals: Module 5

Numerical 1: Bilinear Interpolation (Upsampling)

Given: A low-resolution 2×2 grid with the following corner pixel values that we wish to upsample into a 4×4 grid:

- Top-left: 10 Top-right: 20
- Bottom-left: 30 Bottom-right: 40

Objective: Estimate the inner pixel values using bilinear interpolation.
Steps:

1. **Horizontal Interpolation (Rows):** We have 3 intervals to span between the edges.
   - Top Row (10 to 20): Step size $= (20 - 10)/3 = 3.33$.
   - Values: $10 \rightarrow 13.3 \rightarrow 16.7 \rightarrow 20$.
   - Bottom Row (30 to 40): Step size $= (40 - 30)/3 = 3.33$.
   - Values: $30 \rightarrow 33.3 \rightarrow 36.7 \rightarrow 40$.
2. **Vertical Interpolation (Columns):** Now interpolate the middle columns vertically. The step size is $(Bottom - Top)/3$.
   - Column 2 (13.3 to 33.3): Step size $= (33.3 - 13.3)/3 = 6.67$.
   - Values: $13.3 \rightarrow 20.0 \rightarrow 26.7 \rightarrow 33.3$.
   - Column 3 (16.7 to 36.7): Step size $= (36.7 - 16.7)/3 = 6.67$.
   - Values: $16.7 \rightarrow 23.3 \rightarrow 30.0 \rightarrow 36.7$.

Interpretation: The resulting 4×4 interpolated image matrix is smoothly transitioned:

10.0	13.3	16.7	20.0
16.7	20.0	23.3	26.7
23.3	26.7	30.0	33.3
30.0	33.3	36.7	40.0

Numerical 2: IoU and Dice Coefficient

Given: In an image segmentation task, we have the Ground Truth mask (A) and the Model's Predicted mask (B):

- Ground Truth area ($|A|$) = 50 pixels
- Predicted area ($|B|$) = 60 pixels
- Intersection (correctly predicted pixels, $|A \cap B|$) = 40 pixels

Objective: Calculate the Intersection over Union (IoU) and the Dice Coefficient.
Steps:

1. **Intersection over Union (IoU):**

$$\text{Union } |A \cup B| = |A| + |B| - |A \cap B|$$

$$|A \cup B| = 50 + 60 - 40 = 70 \text{ pixels}$$

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|} = \frac{40}{70} \approx 0.571 \text{ (or } 57.1\%)$$

2. **Dice Coefficient (F1 Score):**

$$\text{Dice} = \frac{2|A \cap B|}{|A| + |B|}$$

$$\text{Dice} = \frac{2 \times 40}{50 + 60} = \frac{80}{110} \approx 0.727 \text{ (or } 72.7\%)$$

Interpretation: Both metrics evaluate segmentation accuracy. IoU penalizes false positives and false negatives more heavily than Dice, which is why IoU (0.57) is lower than Dice (0.72).

3 Module 6: Action Recognition and Object Tracking

3.1 Video Understanding Basics

- **Video vs. Image**: An image has only spatial information (width, height). A video adds a **temporal** dimension (x, y, t).
- **Why Temporal Matters**: A single frame of a person with hands together could be praying, clapping, or resting. A sequence of frames defines the action.
- **Action Recognition**: "What is happening?" (e.g., running, fighting).
- **Object Tracking**: "Where is it happening over time?" (Locating an object continuously across frames).

3.2 Motion Capture and Modeling

- **Marker-based**: Uses reflective markers on joints and infrared cameras. Highly accurate, used in biomechanics and animation.
- **Markerless**: Uses AI/Pose-estimation (e.g., OpenPose, MoveNet) to detect joints from standard video. Non-invasive.
- **Kinematic Models (How things move, ignoring force)**:
  1. **Point Mass Model**: Treats object as a single point. Ignores shape, size, and rotation.
  2. **Rigid Body Model**: Accounts for shape and rotation, assuming the object doesn't deform.
  3. **Articulated Body Model**: Multiple rigid bodies connected by joints (e.g., a human body or robotic arm).
- **Dynamic Models**: Examines *why* things move, analyzing mass, forces, and torques.

3.3 Optical Flow

- **Definition**: The pattern of apparent motion of image objects between two consecutive frames caused by the movement of object or camera.
- **Brightness Constancy Assumption**: The intensity (color/brightness) of a pixel representing a moving object remains constant between frames; only its position (x, y) changes.

$$I_x u + I_y v + I_t = 0$$

  where I_x, I_y are spatial gradients, I_t is temporal gradient, and (u, v) is the optical flow vector.
- **The Aperture Problem**: Viewing motion through a small region makes it impossible to determine the true direction of motion along an edge. Therefore, a single pixel is not enough to calculate optical flow (we have 1 equation but 2 unknowns: u and v).
- **Solutions**:
  - **Lucas-Kanade Method**: Assumes *Local Motion Consistency* (nearby pixels in a small window move in the same way). Produces **Sparse Optical Flow** (only tracks key feature points/corners).

  - **Horn-Schunck Method**: Assumes *Global Smoothness* (motion changes smoothly across the entire image). Produces **Dense Optical Flow** (calculates motion vectors for every single pixel).

3.4 Tracking Methods & Case Study

- **Kalman Filter**: A predictive model that estimates an object's future position based on its past trajectory and updates the prediction using new measurements. Fast and efficient for linear motion.
- **Particle Filter**: Used for complex, non-linear motion tracking.

Case Study: CCTV Surveillance System

Scenario: A camera monitors a shopping mall to ensure safety and security.
Q1: How will the system follow a person across frames?

- **Detect**: Identify the person in the initial frame using object detection.
- **Track**: Estimate the person's position in subsequent frames based on appearance and motion models.
- **Update**: Continuously update the person's location vector as they move.

Q2: How will the system detect suspicious behavior (e.g., fighting)?

- **Analyze**: Monitor the sequence of spatio-temporal movements across consecutive frames.
- **Identify**: Detect unusual or abnormal motion patterns that deviate from normal walking behavior.
- **Classify**: Use an action recognition classifier to tag the abnormal movement sequence as "fighting".

3.5 3D Vision & Depth Estimation

- **Stereo Vision**: Uses two cameras (Left and Right) to estimate depth, mimicking human eyes.
- **Stereo Matching**: Finding the exact same physical point in both the left and right images.
- **Disparity**: The difference in the pixel coordinates of the same point in the two images.
- **Depth Calculation**: Depth is **inversely proportional** to disparity.
  - *Large disparity = Object is close.*
  - *Small disparity = Object is far away.*

3.6 Solved Numericals: Module 6

Numerical 1: Optical Flow (Brightness Constancy)

Given: A tracking system observes a pixel moving between two consecutive frames. The image gradients at the pixel are calculated as follows:

- Spatial gradient in X-direction (I_x) = 2
- Spatial gradient in Y-direction (I_y) = 3
- Temporal gradient (I_t) = -12

It is known that the object is moving **purely horizontally** (meaning the vertical velocity component $v = 0$).
Objective: Find the horizontal velocity u of the pixel.
Steps:

1. **Formula:** State the Brightness Constancy Equation:
$$I_x u + I_y v + I_t = 0$$
2. **Substitute values:**
$$(2)u + (3)(0) + (-12) = 0$$
3. **Solve for u:**
$$2u - 12 = 0 \implies 2u = 12 \implies u = 6$$

Interpretation: The optical flow vector is $(u, v) = (6, 0)$. This means the pixel moved 6 pixels to the right between the two frames.

Numerical 2: Depth Estimation using Stereo Vision

Given: A stereo vision system has two parallel cameras:

- Focal length (f) = 50 mm
- Baseline distance between cameras (B) = 120 mm
- An object is detected at $x_L = 400$ in the left image and $x_R = 390$ in the right image (in pixels).
- Sensor pixel size = 0.01 mm/pixel

Objective: Calculate the depth (Z) of the object from the cameras.
Steps:

1. **Calculate Disparity in pixels:**

$$d_{\text{pixels}} = x_L - x_R = 400 - 390 = 10 \text{ pixels}$$

2. **Convert Disparity to Physical Units (mm):**

$$d = d_{\text{pixels}} \times \text{pixel size} = 10 \times 0.01 = 0.1 \text{ mm}$$

3. **Calculate Depth using Triangulation Formula:**

$$Z = \frac{f \cdot B}{d}$$

$$Z = \frac{50 \text{ mm} \times 120 \text{ mm}}{0.1 \text{ mm}}$$

$$Z = \frac{6000}{0.1} = 60,000 \text{ mm}$$

4. **Convert to standard units:**

$$60,000 \text{ mm} = 60 \text{ meters}$$

Interpretation: The object is 60 meters away from the stereo camera setup. This demonstrates the inverse relationship: a small disparity (0.1 mm) corresponds to a large depth (60 m).